PENGUIN FISH CATCH GAME DEVELOPMENT USING PYTHON

AIM:

To design and develop an interactive 2D Penguin Fish Catch Game using Python Tkinter that allows the player to control a penguin using the left and right arrow keys, start the game using the spacebar, catch falling fish to increase the score, and avoid falling ice blocks. The game includes collision detection, lives tracking, win and game-over detection, and restart and exit options.

PROCEDURE:

1. Start the program and import the required Python libraries: Tkinter and random.
2. Create the main game window using Tkinter and set its title to “PENGUIN FISH CATCH”.
3. Design the graphical interface using labels, buttons, frames, and a Canvas widget.
4. Define the game settings and draw the penguin, snowy hills, and ground inside the game area using Canvas shapes (ovals, polygons, and rectangles).
5. Initialize the score to 0 and the player's lives to 3.
6. Implement keyboard controls using the left and right arrow keys to move the penguin within the game area.
7. Use the spacebar to start the game so that fish and ice blocks begin to fall.
8. Generate falling fish and ice blocks at random positions using the random module, and update their positions continuously using Tkinter's after() method to create animation.
9. Implement collision detection between the penguin and each falling object; remove objects that reach the ground.

10. Increase the score by 10 points whenever the penguin catches a fish and update the score display. Display “YOU WIN!” when the score reaches 200.

11. Reduce the player's lives when the penguin is hit by an ice block. Display “GAME OVER” when all lives are lost.

12. Provide Restart Game and Exit buttons, execute the program, and verify the game controls, scoring, collision detection, and restart functionality.

PROGRAM CODE:

```
import tkinter as tk
import random

# Game Settings
WIDTH = 600
HEIGHT = 450
GROUND_Y = HEIGHT - 40
PENGUIN_W = 50
PENGUIN_H = 62
PENGUIN_SPEED = 25
PENGUIN_Y = GROUND_Y - PENGUIN_H
ITEM_SIZE = 26
FISH_SPEED = 4
ICE_SPEED = 6
SPAWN_CHANCE = 0.04
WIN_SCORE = 200
DELAY = 20          # about 50 frames per second


class PenguinGame:
    def __init__(self, root):
        self.root = root
        self.root.title("PENGUIN FISH CATCH")
        self.root.resizable(False, False)
        self.root.configure(bg="#082F49")

        tk.Label(
            root,
            text="PENGUIN FISH CATCH",
            font=("Arial Black", 22),
            bg="#082F49",
            fg="white"
        ).pack(pady=10)

        info_frame = tk.Frame(root, bg="#082F49")
        info_frame.pack()
```

```
self.score_label = tk.Label(
    info_frame,
    text="SCORE: 0",
    font=("Arial", 14, "bold"),
    bg="#082F49",
    fg="#FACC15"
)
self.score_label.pack(side="left", padx=30)

self.lives_label = tk.Label(
    info_frame,
    text="LIVES: 3",
    font=("Arial", 14, "bold"),
    bg="#082F49",
    fg="#86EFAC"
)
self.lives_label.pack(side="left", padx=30)

self.canvas = tk.Canvas(
    root,
    width=WIDTH,
    height=HEIGHT,
    bg="#7DD3FC",
    highlightthickness=2,
    highlightbackground="#E0F2FE"
)
self.canvas.pack(padx=20, pady=10)

button_frame = tk.Frame(root, bg="#082F49")
button_frame.pack(pady=10)

tk.Button(
    button_frame,
    text="RESTART GAME",
    font=("Arial", 12, "bold"),
    bg="#22C55E",
    fg="black",
    command=self.restart
).pack(side="left", padx=10)

tk.Button(
    button_frame,
    text="EXIT",
    font=("Arial", 12, "bold"),
    bg="#EF4444",
    fg="white",
    command=root.destroy
).pack(side="left", padx=10)

root.bind("<KeyPress>", self.on_key_press)
```

```
    self.timer = None
    self.restart()

def restart(self):
    if self.timer is not None:
        self.root.after_cancel(self.timer)
        self.timer = None
    self.score = 0
    self.lives = 3
    self.running = True
    self.started = False
    self.penguin_x = (WIDTH - PENGUIN_W) // 2
    self.items = []          # each item = [x, y, kind]
    self.score_label.config(text="SCORE: 0")
    self.lives_label.config(text="LIVES: 3")
    self.draw_game()
    self.update_game()

def on_key_press(self, event):
    if event.keysym in ("r", "R"):
        self.restart()
        return
    if not self.running:
        return
    if event.keysym == "Left":
        self.penguin_x = max(0, self.penguin_x - PENGUIN_SPEED)
    elif event.keysym == "Right":
        self.penguin_x = min(WIDTH - PENGUIN_W,
                             self.penguin_x + PENGUIN_SPEED)
    elif event.keysym == "space":
        self.started = True

def spawn_item(self):
    x = random.randint(0, WIDTH - ITEM_SIZE)
    kind = "fish" if random.random() < 0.7 else "ice"
    self.items.append([x, -ITEM_SIZE, kind])

def update_game(self):
    if not self.running:
        return

    if self.started:
        if random.random() < SPAWN_CHANCE:
            self.spawn_item()

        for item in self.items[:]:
            x, y, kind = item
            item[1] += FISH_SPEED if kind == "fish" else ICE_SPEED
            y = item[1]
            # Collision between the penguin and a falling object
```

```
            if (
                x < self.penguin_x + PENGUIN_W and
                x + ITEM_SIZE > self.penguin_x and
                y + ITEM_SIZE > PENGUIN_Y and
                y < PENGUIN_Y + PENGUIN_H
            ):
                self.items.remove(item)
                if kind == "fish":
                    self.score += 10
                    self.score_label.config(text=f"SCORE: {self.score}")
                    if self.score >= WIN_SCORE:
                        self.game_over("YOU WIN!")
                        return
                else:
                    self.lives -= 1
                    self.lives_label.config(text=f"LIVES: {self.lives}")
                    if self.lives == 0:
                        self.game_over("GAME OVER")
                        return

            # Object reached the ground
            elif y > GROUND_Y:
                self.items.remove(item)

    self.draw_game()
    self.timer = self.root.after(DELAY, self.update_game)

def draw_penguin(self):
    x = self.penguin_x
    y = PENGUIN_Y
    c = self.canvas
    # flippers
    c.create_polygon(x + 4, y + 30, x - 8, y + 48, x + 10, y + 46,
                     fill="#111827", outline="")
    c.create_polygon(x + 46, y + 30, x + 58, y + 48, x + 40, y + 46,
                     fill="#111827", outline="")
    # feet
    c.create_oval(x + 6, y + 56, x + 24, y + 64,
                  fill="#F97316", outline="")
    c.create_oval(x + 26, y + 56, x + 44, y + 64,
                  fill="#F97316", outline="")
    # body and belly
    c.create_oval(x, y + 12, x + 50, y + 62,
                  fill="#111827", outline="")
    c.create_oval(x + 10, y + 24, x + 40, y + 60,
                  fill="white", outline="")
    # head
    c.create_oval(x + 7, y, x + 43, y + 30,
                  fill="#111827", outline="")
    # eyes
```

```
    c.create_oval(x + 15, y + 8, x + 24, y + 17,
                  fill="white", outline="")
    c.create_oval(x + 26, y + 8, x + 35, y + 17,
                  fill="white", outline="")
    c.create_oval(x + 18, y + 10, x + 22, y + 14,
                  fill="black", outline="")
    c.create_oval(x + 28, y + 10, x + 32, y + 14,
                  fill="black", outline="")
    # beak
    c.create_polygon(x + 19, y + 19, x + 31, y + 19, x + 25, y + 28,
                     fill="#FB923C", outline="")

def draw_fish(self, x, y):
    c = self.canvas
    c.create_oval(x, y + 5, x + 20, y + 21,
                  fill="#F97316", outline="#7C2D12")
    c.create_polygon(x + 18, y + 13, x + 27, y + 4, x + 27, y + 22,
                     fill="#F97316", outline="#7C2D12")
    c.create_oval(x + 4, y + 10, x + 8, y + 14,
                  fill="white", outline="")

def draw_ice(self, x, y):
    s = ITEM_SIZE
    self.canvas.create_polygon(
        x + s // 2, y, x + s, y + s // 2,
        x + s // 2, y + s, x, y + s // 2,
        fill="#F0F9FF", outline="#1D4ED8", width=2
    )

def draw_game(self):
    c = self.canvas
    c.delete("all")

    # Background hills and snowy ground
    c.create_polygon(0, GROUND_Y, 120, 250, 240, GROUND_Y,
                     fill="#BAE6FD", outline="")
    c.create_polygon(300, GROUND_Y, 450, 220, 600, GROUND_Y,
                     fill="#BAE6FD", outline="")
    c.create_rectangle(0, GROUND_Y, WIDTH, HEIGHT,
                       fill="#F8FAFC", outline="")

    # Falling objects
    for x, y, kind in self.items:
        if kind == "fish":
            self.draw_fish(x, y)
        else:
            self.draw_ice(x, y)

    self.draw_penguin()
    if not self.started and self.running:
```

```
            c.create_text(
                WIDTH // 2, 120,
                text="Press SPACE to start",
                font=("Arial", 14, "bold"),
                fill="#0C4A6E"
            )
            c.create_text(
                WIDTH // 2, 148,
                text="Catch the fish, avoid the ice!",
                font=("Arial", 12),
                fill="#0C4A6E"
            )

    def game_over(self, message):
        self.running = False
        self.timer = None
        self.draw_game()
        colour = "#EF4444" if message == "GAME OVER" else "#FACC15"
        self.canvas.create_rectangle(
            130, 130, 470, 260,
            fill="#082F49",
            outline="#E0F2FE",
            width=2
        )
        self.canvas.create_text(
            WIDTH // 2, 170,
            text=message,
            font=("Arial Black", 26),
            fill=colour
        )
        self.canvas.create_text(
            WIDTH // 2, 225,
            text=f"Final Score: {self.score} (press R to restart)",
            font=("Arial", 13),
            fill="white"
        )


# Main Application
root = tk.Tk()
game = PenguinGame(root)
root.mainloop()
```

OUTPUT:

#INITIAL GAME



#GAME PLAY



#GAME OVER



#YOU WIN



RESULT:

Thus, an interactive 2D Penguin Fish Catch Game was developed using Python Tkinter with a graphical game interface, keyboard-based penguin movement, random falling fish and ice blocks, collision detection, score calculation, lives tracking, win and game-over handling, restart functionality, and exit controls. The program demonstrates event-driven programming, GUI design, animation, collision detection, conditional statements, and basic game development concepts.